МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-67

Выполнил
К. В. Юреня,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов классификации, таких как
метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 10

Задание:

Вариант 10

- Adult Census Income
- Предсказать, превышает ли доход человека $50 тыс. в год
- **Задания:**
  1. Загрузите данные, обработайте пропуски и категориальные признаки;
  2. Разделите данные на обучающую и тестовую выборки;
  3. Обучите k-NN, Decision Tree и SVM;
  4. Сравните модели по метрике `precision` для класса ">50K";
5. Определите, какой алгоритм лучше всего идентифицирует людей с высоким доходом.

Выполнение:

Код программы

```
import pandas as pd
import numpy as np
df = pd.read_csv('adult.csv')
display(df.head())
print("Размер датасета:", df.shape)
```

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report, precision_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv('adult.csv')
df_clean = df.replace('?', np.nan)

X = df_clean.drop('income', axis=1)
```

```
y = df_clean['income'].apply(lambda x: 1 if '>50K' in str(x) else 0)

# Разделение столбцов на числовые и категориальные
num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X.select_dtypes(include=['object']).columns.tolist()

# Пайплайны предобработки (заполнение пропусков, масштабирование, One-Hot Encoding)
num_transformer = Pipeline(
    [('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())]
)

cat_transformer = Pipeline(
    [
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_transformer, num_cols),
        ('cat', cat_transformer, cat_cols),
    ]
)

# 2. Разделение на обучающую и тестовую выборки (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train_prep = preprocessor.fit_transform(X_train)
X_test_prep = preprocessor.transform(X_test)

# Обучение моделей и исследование k-NN, расчет для >50K
k_values = [3, 5, 7, 11, 15]
results_knn = {}

print('ИССЛЕДОВАНИЕ k-NN')
for k in k_values:
  knn = KNeighborsClassifier(n_neighbors=k)
  knn.fit(X_train_prep, y_train)
```

```
  y_pred = knn.predict(X_test_prep)
  prec = precision_score(y_test, y_pred)
  results_knn[k] = prec
  print(f'k-NN (k={k}) -> Precision (>50K): {prec:.4f}')

# Обучаем k-NN с оптимальным k=11 для дальнейшего отчета
best_knn = KNeighborsClassifier(n_neighbors=11)
best_knn.fit(X_train_prep, y_train)

# Дерево решений (Decision Tree)
dt = DecisionTreeClassifier(max_depth=10, random_state=42)
dt.fit(X_train_prep, y_train)
y_pred_dt = dt.predict(X_test_prep)
prec_dt = precision_score(y_test, y_pred_dt)
print(f'\nDecision Tree Precision (>50K): {prec_dt:.4f}')

# Метод опорных векторов (SVM)
svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train_prep, y_train)
y_pred_svm = svm.predict(X_test_prep)
prec_svm = precision_score(y_test, y_pred_svm)
print(f'SVM Precision (>50K): {prec_svm:.4f}')

# Сводная таблица результатов
summary_data = []
for k, prec in results_knn.items():
  summary_data.append({
      'Модель': 'k-NN',
      'Параметры': f'k = {k}',
      'Precision (>50K)': prec,
  })

summary_data.append({
    'Модель': 'Decision Tree',
    'Параметры': 'max_depth = 10',
    'Precision (>50K)': prec_dt,
})
summary_data.append(
    {'Модель': 'SVM', 'Параметры': 'kernel = rbf', 'Precision (>50K)': prec_svm}
)
```

```
    df_summary = pd.DataFrame(summary_data)
    print('\nСВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ')
    display(df_summary)

    # 5. Детальные отчеты классификации для определения лучшего алгоритма идентификации
    высокого дохода
    print('\n 5. ДЕТАЛЬНЫЙ ОТЧЕТ ДЛЯ КЛАССА ">50K"')
    print('-k-NN (k=11) -')
    print(
        classification_report(
            y_test, best_knn.predict(X_test_prep), target_names=['<=50K', '>50K']
        )
    )

    print('- Decision Tree -')
    print(
        classification_report(
            y_test, dt.predict(X_test_prep), target_names=['<=50K', '>50K']
        )
    )

    print('- SVM -')
    print(
        classification_report(
            y_test, svm.predict(X_test_prep), target_names=['<=50K', '>50K']
        )
    )
```

Результаты работы программы

```
 ИССЛЕДОВАНИЕ k-NN
k-NN (k=3) -> Precision (>50K): 0.6526
k-NN (k=5) -> Precision (>50K): 0.6691
k-NN (k=7) -> Precision (>50K): 0.6848
k-NN (k=11) -> Precision (>50K): 0.6953
k-NN (k=15) -> Precision (>50K): 0.6931

Decision Tree Precision (>50K): 0.7715
SVM Precision (>50K): 0.7502

СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
```

	Модель	Параметры	Precision (>50K)
0	k-NN	k = 3	0.652632
1	k-NN	k = 5	0.669065
2	k-NN	k = 7	0.684822

	Модель	Параметры	Precision (>50K)
3	k-NN	k = 11	0.695335
4	k-NN	k = 15	0.693149
5	Decision Tree	max_depth = 10	0.771503
6	SVM	kernel = rbf	0.750203

```
 5. ДЕТАЛЬНЫЙ ОТЧЕТ ДЛЯ КЛАССА ">50K"
- k-NN (k=11) -
              precision    recall  f1-score   support

       <=50K       0.88      0.92      0.90      4945
        >50K       0.70      0.61      0.65      1568

    accuracy                           0.84      6513
   macro avg       0.79      0.76      0.77      6513
weighted avg       0.84      0.84      0.84      6513

- Decision Tree -
              precision    recall  f1-score   support

       <=50K       0.87      0.95      0.91      4945
        >50K       0.77      0.57      0.65      1568

    accuracy                           0.86      6513
   macro avg       0.82      0.76      0.78      6513
weighted avg       0.85      0.86      0.85      6513

- SVM -
              precision    recall  f1-score   support

       <=50K       0.88      0.94      0.91      4945
        >50K       0.75      0.59      0.66      1568

    accuracy                           0.85      6513
   macro avg       0.81      0.76      0.78      6513
weighted avg       0.85      0.85      0.85      6513
```

Вывод: Сравнение трех моделей на датасете *Adult Census Income* показало, что Дерево решений (`max_depth=10`) лучше всего идентифицирует людей с высоким доходом, достигнув максимальной точности Precision = 77.15%. На втором месте SVM (75.02%), а k-NN ($k=11$) показал наименьшую точность (69.53%) из-за проклятия размерности при кодировании признаков.